# In a single pass, most local 4-bit executors tested transcribe a detailed plan; two of thirteen bind it


## Abstract

We asked one question: can a free, locally hosted, 4-bit-quantised open-weight language model fill the *execution* role in routine genomic analysis when the plan is authored separately? Twelve local and three frontier API executors each generated a run script for per-sample mitochondrial variant calling in a single pass. The design covered nine (plan file, track) cells, which reduce to seven plan conditions, at three seeds: 324 local and 81 frontier cells. A further 443 cells came from reasoning, repeated-sampling and budget passes. Another 312 cells were added after the second review cycle, in a seed-extension pass and a perturbed-plan pass. Both tasks in this study were selected for having no data-dependent parameter choices, so that a plan could state the answer. That criterion bounds every result below. The finding is two-part. First, literal command lines are necessary for most local executors. With no plan, 6 of 108 local runs produced the correct call set, against 13 of 27 frontier runs. The frontier executors were 9 of 9 at the leanest plan file, which contains no command lines at all (exact 95% CI [0.66, 1.00]), and gained nothing from further detail. The local executors reached 34 of 36 only at the most detailed condition, which supplies the literal command lines. Executor class — 4-bit open-weight models of at most 35 B against a commercial API — therefore determined how much specification an executor needed. Quantisation and vendor are confounded with weight availability. The results are consistent with copy-pasteability as the active ingredient. At ten seeds per model, the contrast is measured, not read off the plan files. Two plans add the same `lofreq call-parallel` invocation to the lean plan. The plan that states it as a single runnable line scored 63/127. The other plan states it across six lines that cannot be pasted and run. It also spells out the one detail a model is most likely to get wrong. It scored 31/126 (Fisher exact p = 0.00005 against 63/127). That is no better than the lean plan itself, which scored 9/36. The all-code condition scored 109/127. Stripping every word of prose from the detailed condition produced no change resolvable at these sample sizes. Verbosity is not what varies with outcome: the shortest plan in the set, 159 words of bare code fences, is the second best. A plan supplies text the executor can transcribe, not information the executor lacks. Second, most local executors use that text by transcription, not by binding it to the stated inputs. One perturbation used the detailed plan verbatim, with the reference genome moved to a path stated in the prompt's dataset section. It dropped the class from 34 of 36 perfect runs to 6 of 39, and the outcome is bimodal by model. Eleven of thirteen local models, including the paper's original headline model, copied the plan's stale path verbatim and failed at the first step in all three seeds. Two (`gemma4:31b` and `gpt-oss:20b`) rebound every path and were perfect in all three seeds (each 3/3, 95% CI [0.29, 1.00]). Binding therefore exists in this model class but is rare, and rank on the unperturbed gradient does not predict it. No frontier executor was run on the perturbed plan. With up to three attempts and the error shown after each failure, two copier models became perfect on the second attempt in every seed, and a third in two of three seeds. Eight copiers never recovered. Repair lifted the perturbed class from 6 of 39 perfect runs to 14 of 39. Consistent with transcription, a correct pipeline built with no plan already contains about half of the detailed plan's literal command tokens (0.505 over 13 frontier no-plan successes). A plan-conditioned correct script contains about three quarters (0.749 over 110 local successes). In an uncontrolled case study, two local models drove a 30-step Galaxy workflow on usegalaxy.org to a gene count matching the published benchmark. But the decisive invocation body was supplied literally, two of seven plan steps were performed by a human, and neither executor completed the reporting step; a human read the per-sample counts out of the server. The same reporting job was then put to the same two models as a standalone task, rather than as the last step of that plan. They answered correctly in 118 of 144 sample-level judgements, with 14 correct abstentions and no fabricated value. The omission is therefore a property of task scoping, not of capability. The accompanying fabrication study bounds fabrication at about 12.5% at the session level (0 of 24 scorable sessions). That bound is conditional on an answer being produced at all, which happened in 24 of 30 launches. Under this study's cost model, the local/API crossover falls between roughly 10,000 annualised runs per year and never, depending on an API-side labour term that was not measured. For most local executors, a sufficient plan is therefore one whose command lines are literal *and current*. One moved path separates a working detailed plan from total failure, unless the executor is one of the few that bind, or one of the few that repair when shown the error.

## Introduction

A laboratory that generates its own sequencing data does the same analysis many times. Each batch goes through a pipeline settled months ago and not in question. None of that work is free. Someone has to bind the pipeline to this batch's file names, run it, notice which samples failed, and re-run them.

This paper takes two instances of that problem. The first is per-sample mitochondrial DNA variant calling from paired-end Illumina reads. The second is per-sample RNA-seq quantification against a reference annotation, on a shared public Galaxy server. We ask one narrow question. Given an analysis plan authored once, how much detail does that plan need before a free open-weight language model, running on hardware the laboratory already owns, can carry out the execution? We do not ask whether a language model can design the analysis. We report what a fixed set of models did, on two workflows, on five machines. Human attention is the motivation for this work, and this study did not measure it. The claim that this architecture saves human time is therefore conditional on unsupervised operation. This study did not achieve unsupervised operation, and we do not assert the claim.

The honest baseline deserves stating without hedging. Some pipelines are stable and repeatedly run. For those, a validated shell script under version control, a Snakemake or Nextflow rule set, a CWL description, or a saved Galaxy workflow is simply better than a language model (Di Tommaso et al. 2017; Mölder et al. 2021). Those tools are deterministic, auditable, free to invoke, and need no GPU. A language model earns its keep at the edges of that template. The edges are one-off analyses, and adaptation when layouts or tool versions change. They include a natural-language interface for people who can evaluate a result but do not write code. They also include *binding*: attaching a fixed, already-correct workflow to a specific, messy set of local inputs. Benchmark 2 was designed to exercise binding, and does so only in part (Results, *Case study*). The perturbed-plan pass tests binding directly, and finds it in two of thirteen local models (Results, *A one-path perturbation*).

Separating a specification from a process that carries it out is not a contribution of this paper. In genomics, it is the design of every workflow system in routine use. In those systems, a declarative specification is authored once, and an engine binds it to concrete inputs many times (Di Tommaso et al. 2017; Mölder et al. 2021; Amstutz et al. 2016; Galaxy Community 2024). What differs here is that a language model performs the binding, rather than a deterministic engine. The binding is therefore neither guaranteed nor reproducible. The same split runs through the language-model literature. It appears in plan-and-solve prompting, least-to-most decomposition, ReAct, HuggingGPT, compiler-style graph dispatch and Reflexion (Wang et al. 2023; Zhou et al. 2023; Yao et al. 2023; Shen et al. 2023; Kim et al. 2024; Shinn et al. 2023). Its application to bioinformatics workflow authoring has been surveyed (Alam and Roy 2025). We adopt this pattern; we do not propose it. The contribution claimed is narrow. It is a measurement of that arrangement on genomic work, on hardware a laboratory can buy outright, with plan detail as the independent variable.

In practice, the execution side is supplied by an *agent harness*. Several were in wide use by mid-2026, and no comparison among them was run here. Benchmark 2 used `galaxyproject/loom`, for deployment reasons given in the Supplement. Nothing in the results is known to be harness-independent. Benchmark 1's harness is not an agent harness at all. It is one prompt in, one script out, executed externally, with no loop, no observation and no repair. That isolates plan detail as the only variable, but it means the system should not be called agentic. Benchmark 2 was added to supply the missing loop. The closest bioinformatics work evaluates open-ended tool-using research agents against task suites (Huang et al. 2025; Mitchener et al. 2025; Su et al. 2025; Mehandru et al. 2025). We ran none of those suites, so **no external anchor was obtained** and no number here sits on a published scale. Benchmark 2 has two external comparators, both used with their protocols stated. First, a human executed the same workflow manually on the same server. Second, eight frontier models acting as *both* planner and executor on the same data cost \$2.82–\$131.83 per run (Nekrutenko 2026). That source is a post by the present author, with unpublished plans and no replicates. It bounds API cost and is not a frontier control.

Two properties of open-weight models decide whether one fits on a given machine, and parameter count captures neither. A mixture-of-experts (MoE) model routes each token to a subset of experts. Its per-token compute therefore scales with *active* parameters, while its memory residency scales with *total* parameters. The key–value cache grows with context length, not with model size. It grows more slowly than a naive

projection suggests, because modern models share key and value projections across heads (Table 1).

**Table 1.** The four local models on which residency was measured, plus the API executors. This is a selection, not a catalogue. Fourteen local models appear in this study. The full list, with digests, declared context lengths and per-model default decoding parameters, is Supplementary Table S8. Local models were served by Ollama at 4-bit quantisation. Quantisation is not uniform: `gpt-oss:20b` ships MXFP4, while every other local model is `q4_K_M`. **No residency column is given**, because this study's own residency figures for one of these models do not reconcile. Every residency measurement made, with the discrepancy stated inline, is Supplementary Table S4. Residency was measured for four of the fourteen models, and not for the other ten. Filling that column costs roughly six minutes of cold load per model. That was not done, and no figure is estimated. Declared parameter counts span 4.0 B (`nemotron-3-nano:4b`) to 36.0 B (`qwen3.6:35b-a3b`).

| Executor | Architecture | Total params | Active params/token | Quantisation | Role |
|---|---|---|---|---|---|
| `qwen3.6:27b` | dense | 27 B | 27 B | q4_K_M | benchmark 1; cross-platform arm; error matrix |
| `qwen3.8:27b` | dense | 27 B | 27 B | q4_K_M | out-of-sample addition; benchmark 2 dense arm |
| `qwen3.6:35b-a3b` | sparse MoE | 35 B | 3 B | q4_K_M | benchmark 1; benchmark 2 MoE arm |
| `gemma4:12b` | dense | 12 B | 12 B | q4_K_M | throughput measurement only |
| `claude-opus-5`, `claude-sonnet-5`, `claude-haiku-4-5` | commercial API | — | — | — | frontier reference gradient |
| `claude-opus-4-7`, `claude-sonnet-4-6`, `claude-haiku-4-5` | commercial API | — | — | — | error matrix |

Two workflow classes were run: alignment and small-variant calling, at 7 measured plan steps, and RNA-seq quantification, at 30 measured workflow steps. Four further classes cover most of the remaining routine per-sample work: single-cell RNA-seq, de novo assembly, metagenomic profiling, and ChIP-seq or ATAC-seq peak calling. None of the four was run. Which two classes were run is the single most important limitation in this paper. **The two classes run here are the two in which the plan can state the answer, and the four not run are ones in which it cannot**. In each of the four, at least one parameter is a scientific judgement made against the data: filtering thresholds, clustering resolution, assembler parameterisation, database version, peak-calling thresholds. For those classes, there is no canonical value for a plan to carry, and no invocation for an executor to transcribe. The plan-sufficiency result is therefore conditional on tasks with no data-dependent parameter choices. It may be in part a restatement of the task-selection criterion (Supplementary Table S19).

Two declarations bound everything below. The plan gradient is exploratory, because the most detailed plan condition was written after observing how the leaner conditions failed. Benchmark 2 is a case study, and no claim is supported by pooling the two. One confound the design cannot remove belongs before any result. At the detailed end of the gradient, the plan was *specified* to be copy-pasteable. The v2 meta-prompt instructs the planner to emit "every flag, with its exact value". It states the assumption that "the implementer will copy-paste your invocations into a shell script". A model that succeeds at v2 is therefore doing at least partly what the condition was designed to require. We measure how much of that is transcription rather than conceding the point, twice. The indirect measure is a token-recall index (Results, *How much of the plan is copied*). The direct measure executes the same plan against inputs its literal paths no longer match (Results, *A one-path perturbation separates transcription from binding*).

## Methods

Extended methods are in the Supplement.

### Study design

Two benchmarks share a thesis but not a harness, a model set, a scoring scheme or an infrastructure (Figure 1). In both, a plan was authored once, off the executor machine, by a frontier model. A free, locally hosted, 4-bit-quantised open-weight model then executed it. Benchmark 1 was single-pass by construction: one prompt in, one bash script out, executed and scored with no opportunity to observe the outcome or retry. Benchmark 2 was multi-turn. Its executor issued tool calls against a live public server, read invocation and job state back, and carried state across session boundaries.

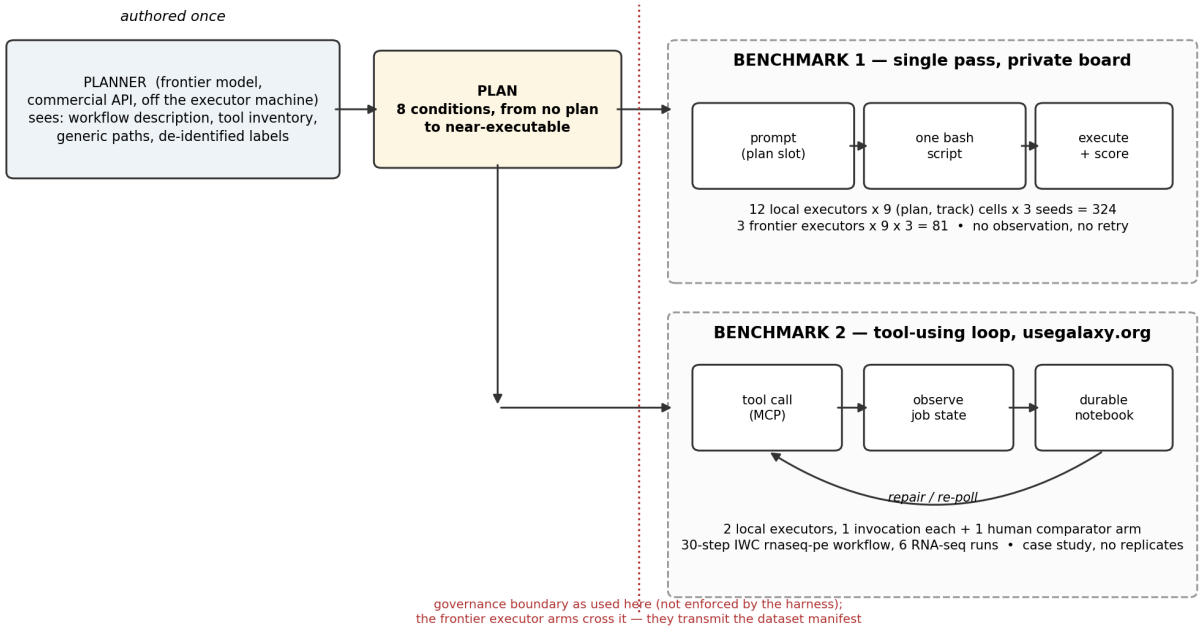


Figure 1: Study design

**Figure 1.** Study design. A plan is authored once, off the executor machine, by a frontier model in the planner role. Benchmark 1 executes that plan in a single pass: one prompt in, one bash script out, executed and scored with no observation step. Benchmark 2 executes it in a tool-using loop against the public usegalaxy.org server, with observation, durable state and repair. The dotted line marks the data-governance boundary as it was used here. The harness does not enforce that boundary. The frontier executor arms of benchmark 1 cross it, because their prompts carry the dataset manifest.

The benchmark-1 design multiplies out exactly. 12 local executors × 9 (plan file, track) cells × 3 seeds = 324 generations, and 3 frontier executors × 9 × 3 = 81. The nine cells reduce to seven plan conditions, because Track B occupies three of them. Across ten released log files, the study ran 848 cells and 75.27 h of generation. Of that, 81 cells and 1.55 h are API calls (Supplementary Tables S1–S2). Two passes run after the second review cycle add 312 local cells in two further log files: a seed extension (`matrix_jetson_seedext.jsonl`, 273 cells) and a perturbed-plan pass (`matrix_jetson_perturbed.jsonl`, 39 cells). Those two passes post-date the ledger above and are not part of it. All cells came from a single inference configuration. 400 earlier local runs made under a quantised (`q8_0`) key–value cache were discarded rather than pooled. Generation outcomes for every block are Supplementary Table S10.

**Benchmark 1: task, plan conditions, and execution**

The task was per-sample mitochondrial variant calling on four deeply downsized paired-end Illumina amplicon samples. The samples come from blood and cheek tissue of a mother–child pair, with nine truth variants per run. The reference workflow indexes the 16,569-bp chrM reference with `bwa index` and `samtools faidx`. It aligns with `bwa mem` piped to `samtools sort`, and indexes the BAM. It calls variants with `lofreq call-parallel`, then compresses and indexes the VCF with `bgzip` and `tabix`. It finishes with a `bcftools query` plus `awk` fan-in that collapses all four samples into one `collapsed.tsv`. Only the fan-in couples samples to one another (Supplementary Figure S1).

Executors were run under eight conditions, spanning no plan at all to a near-executable recipe (Table 2). Six conditions are plan files; Track B and v0.5 are prompt-template variants that substitute no plan file. Three planner meta-prompts produced the plan files. The meta-prompts are reproduced verbatim in the Supplement, with the executor prompt template. Two pieces of content that the results depend on originated in the human meta-prompt, not with the planner model. The first is the literal read-group string, which `PLANNER_PROMPT.md` requires reproduced "EXACT[ly], character-for-character"; it is therefore present in the *lean* v1 condition. The second is the naming of `lofreq call-parallel`, `bgzip` plus `tabix` and `bcftools query`. **The gradient is therefore in part a gradient in how much literal syntax was demanded of the planner, not purely in model-authored detail**. The exception to model authorship is v1g. Its `lofreq` block was extracted mechanically from the Galaxy IUC tool registry (tools-iuc commit `39e7456`). It is a check on sensitivity to plan authorship, not an arm of a planner comparison.

**Table 2.** Plan conditions. Sizes are bytes of the substituted plan text, or of the prompt-template variant for the two no-plan conditions.

| Condition | Size | Summary | Hypothesis tested |
|---|---|---|---|
| Track B | 1.2 KB | Problem statement and tool inventory only | How much does a model do unaided? |
| v0.5 | 1.4 KB | Track B plus one line giving the tool order | Does sequencing alone help? |
| v1 (lean) | 3.1 KB | Numbered bullets naming tools and key flags; no code | Reference lean plan |
| v1.25 | 3.1 KB | v1 plus one literal `lofreq call-parallel` command line | Does one full command bridge v1 to v2? |
| v1.5 | 1.3 KB | v2 with all prose stripped; code fences only | Does the prose change outcomes, or is it decoration? |
| v1g | 4.2 KB | v1 plus a `lofreq` invocation extracted mechanically from the Galaxy IUC tool registry | Can a tool registry substitute for a plan author? |
| v2 (detailed) | 4.6 KB | Exact command line and a Gotchas block per step | Reference detailed plan |
| v2_defensive | 6.5 KB | v2 plus `try()` helper, per-step validation, retry-once, failure log | Does error-handling prose change runtime behaviour? |

**Every Track-B run is a no-plan run regardless of which plan file was nominally passed on the command line**. Pooling by plan file without honouring this yields the convincing but false conclusion that the most detailed plan performs worst. `revision/scripts/gradient.py` implements the correct mapping and asserts it.

All local generations were served by ollama 0.32.5 at `temperature = 0.2` with `num_ctx = num_predict = 16384`. Neither the local seed nor the API run identifier gives bitwise determinism. All claims of reproducible sampling have been removed. **A confound, not merely a disclosure:** the harness overrode only `temperature`. It left `top_k` and `top_p` at each model's own defaults. Those defaults are not uniform across families, and are absent altogether for `qwen3.8:27b` (Supplementary Table S8). Every cross-model

comparison here therefore compares models decoded from different distributions. The gradient itself is a within-model comparison and is unaffected.

Generated scripts were executed on the model host, in a per-cell working directory, under a pinned conda environment, and killed at 600 s. Execution had no human review, and no container, VM, seccomp profile, namespace, cgroup quota or egress restriction. Only the wall-clock kill, the PATH-restricted inventory and the working directory are enforced. The constraints on absolute paths, package managers and network fetches are prompt instructions a model is free to violate. The violation rate is measured in the Results rather than asserted. **The local arm does not meet the sandboxing bar a production deployment needs and is not a deployment template**. Attacker- or accident-controlled text reached the executor through at least five unsanitised channels. No indirect-prompt-injection test was run.

### Scoring, error injection, and interval estimates

Success was assessed on a three-rung ladder matching the released scorer's field names. **M1** (`m1_executes`): `bash run.sh` returned 0 within 600 s. **M2** (`m2_schema`): every expected output exists and `bcftools` accepts every VCF. **M3** (`m3_jaccard`): the per-sample variant output matches the answer key. Two properties of M3 govern how the Results read. **M3 depends on exit status**: `score_run.py` sets it to 0.0 whenever the run exited non-zero, regardless of what is on disk. **M3 is the tolerant Jaccard overlap**, not a per-sample success fraction. It is the per-sample Jaccard of (chrom, pos, ref, alt) against the answer key, with allele frequencies matched within $\pm 0.02$, averaged over four samples. On the 324 local reasoning-off cells it takes only 0.0 and 1.0; on the frontier arm it does not. Conventional variant-calling metrics were computed alongside, not instead (Supplementary Table S12). The departures from the nearest published conventions (Krusche et al. 2019) are enumerated in Supplementary Table S5.

Tool failures were injected with PATH shims applied only to the targeted tool. The generated script saw nothing but ordinary exit codes, stderr and output files. Each executor–plan combination comprises 39 cells: 8 pattern–target combinations injecting a true error $\times$ 3 seeds, 4 injecting no error $\times$ 3 seeds, and 3 uninjected controls. **Two of the seven patterns do not inject an error at all**. `slow_tool` sleeps 30 s and then succeeds; `stderr_warning_storm` emits 200 warnings and then succeeds. A third of the injected matrix therefore tests latency and log-noise tolerance, not recovery. Results are reported by class and never pooled (Supplementary Table S6).

The full gradient was run at $n = 3$ seeds per cell. The headline condition (plan v2, Track A, reasoning off) was raised to $n = 10$ by adding seven seeds and pooling. After the second review cycle, the three discriminating conditions — v1g, v1.25 and v1.5, Track A, reasoning off — were likewise raised to ten seeds per model, across thirteen local executors. The thirteen are the twelve gradient models plus `qwen3.8:27b`. That model's original three seeds sit in a separate log and are not pooled, so it contributes seven. One v1g cell produced no score record and is excluded. The denominators are therefore 126 for v1g and 127 for v1.25 and v1.5 (Supplementary Table S16). This is **repeated sampling from a single configuration, not replication**. Ten seeds on one machine, one engine build, one quantisation, one plan and one scorer vary the sampler and nothing else. One exclusion rule governs both arms. **Cells whose generation ended in truncation, refusal or provider error are scored 0 and retained. No cell was dropped for producing a bad answer**. All success proportions carry Clopper–Pearson exact 95% confidence intervals. The intervals are wide here: 3/3 gives [0.29, 1.00], 10/10 gives [0.69, 1.00].

### The perturbed-plan condition

One condition was added after the second review cycle to separate transcription from binding directly. The plan is v2 **verbatim** — not one character changed. The sandbox holds the same four read pairs, but the reference genome sits at `data/ref/GRCh38_chrM/rCRS.fa`. The plan's literal path, `data/ref/chrM.fa`, does not exist. The prompt's DATASET section states the real path, so every fact needed to succeed is in the executor's context. A script that copies the plan's command lines fails at the first step. A script that binds the plan's commands to the stated inputs succeeds. The sandbox was validated before any model ran, with a hand-written copier script (fails at `bwa index`) and a hand-written binder script (produces the correct call set). Thirteen local executors — the twelve gradient models plus `qwen3.8:27b` — ran at 3 seeds

each, reasoning off, Track A. That is 39 cells (`matrix_jetson_perturbed.jsonl`; per-cell artifacts under `revision/runs_perturbed/`). The control is the unperturbed published v2 condition: 34 of 36 perfect across the twelve gradient models. No frontier executor was run on the perturbed plan, because no API spend was authorised for this pass.

**Benchmark 2 and the fabrication study**

The second benchmark re-quantified *Candidozyma auris* RNA-seq from Santana et al. (2023), BioProject PRJNA904261, six paired-end runs SRR22376027–SRR22376032. The target was a published benchmark with known expected values (~5,594 genes, ~92% assigned). Relative to benchmark 1, it varies four things at once: a second workflow class, a shared production server, a 30-step workflow, and an external cost comparator (Nekrutenko 2026). The executor-selection rule is invoked repeatedly, so it is stated verbatim:

> *take every local model that scored 3/3 at plan v2 Track A with reasoning off in the benchmark-1 gradient, and from that set take the largest dense model and the largest mixture-of-experts model by total parameter count.*

The rule returns `qwen3.6:27b` and `qwen3.6:35b-a3b`. The dense arm actually run was `qwen3.8:27b`, substituted at matched parameter count after the rule was fixed. **That arm is therefore exploratory rather than confirmatory**. Plan sufficiency was never confirmatory on either benchmark, because the Galaxy plan was corrected repeatedly while benchmark 2 ran.

Both executors ran on one Jetson AGX Orin 64 GB, with reasoning suppressed, under `galaxyproject/loom`. Galaxy was reached over MCP, and `notebook.md` served as durable inter-session state. The plan was a router: a 50-line index plus seven step files, so the executor loads only the step it is on. The plan had seven steps, from importing the workflow by TRS identifier to reporting the per-sample counts. **No number reported for this benchmark comes from an agent's own report**. Every state, parameter and count was read back from the Galaxy API or from dataset contents. The reason: one executor produced a fully formed failure report for an event that never occurred. Scoring had four components:

1. Invocation correctness, field by field.
2. Biological outcome against the published benchmark, read from dataset contents.
3. End-to-end reporting: did the executor itself deliver the per-sample table, with a dataset identifier beside every number?
4. A defect ledger attributing every human intervention to the planner or the executor side.

One change belongs in Methods because it changed the design. The original step-7 verification file stated its own expected values. That let an executor that never opens a dataset verify by recitation. The file was rewritten to state no expected values and to sanction `UNAVAILABLE`. The general rule: **any agent evaluation that places the expected answer in the task description cannot distinguish execution from recitation**.

The fabrication incident was a single observation, so a controlled follow-up bounded the rate. The design was 3 conditions × 2 models × 5 seeds, one fresh workspace per cell, a fixed prompt and known ground truth. The task was to fetch six featureCounts summary datasets from a real Galaxy history and report each `Assigned` value with the dataset identifier it was read from. The conditions were **blind** (nothing said about expected values), **leaky** (the expected range stated) and **blocked** (two identifiers replaced with same-shaped ones that do not exist, so `UNAVAILABLE` is the only correct answer). Answers were scored `CORRECT`, `WRONG` — the fabrication category — `ABSTAINED` or `ABSENT`. The design is **30 launches**. 24 launches produced a scorable answer file, giving 144 sample-level judgements; **six, 20%, produced no answer to score**. Three limitations govern the rate. Judgements within a session are not independent, so the session is the unit of analysis. Unscorable launches are excluded, so the rate is fabrication **conditional on an answer having been produced**; the excluded class is structurally the one the original incident belongs to. And every cell is single-shot with a working tool path, whereas the incident arose in a resumed session with no reachable path.

### Cost model, data governance, and availability

Two of the cost model's eight inputs are measured. Frontier-API cost was $0.0662 per run over 81 API cells. Jetson **wall-clock** time was 86.7 s per run; that includes model loading, execution and scoring, against 80.2 s of generation alone. The other six inputs are assumptions, exposed as parameters in `revision/scripts/cost_model.py` (Supplementary Table S9). The model reports a like-for-like **annualised** comparison, with an API-side labour term exposed rather than set to zero. Supervision labour is not priced at all, and benchmark 2 shows it is not zero. **The cost model is therefore the zero-touch case, which benchmark 2 did not achieve**.

The split also defines a data-governance boundary, and **it was respected by the planner role only**. The frontier *executor* arms — 315 API cells in all — transmitted the executor prompt. That prompt carries the dataset manifest with this study's own subject labels. `revision/scripts/payload_audit.py` reconstructs every outbound payload and scans it against a deny-list of local-identity tokens (Supplementary Table S3). Benchmark 2's MCP traffic is neither archived nor reconstructable, and it is outside the audit.

The repository at https://github.com/nekrut/LLM-eval-paper is released under an MIT license. It contains the plan files and prompt templates, the planner meta-prompts, the harness and sweep drivers, the scoring code, every archived `run.sh` and per-cell artifact, the error-matrix logs, and the revision analysis scripts. Four items are **not** released: the Galaxy plan files, the fabrication study's code and per-cell outcomes, the eleven-item defect ledger with its diffs, and the `loom` session logs. Benchmark 2's per-sample counts and gene count are re-readable from public Galaxy histories. Its defect count and attribution, its tool-call and wall-clock figures, and the fabrication study's cell counts rest on `revision/galaxy_demo/FACTS.md`. That file is a hand-typed contemporaneous record of live API read-back — **the same evidentiary class as an assertion in this manuscript** — and those figures are marked as such wherever used.

## Results

### Where the plan is thin, executor class decides

With no plan at all, **6 of 108 local runs produced the correct call set** (exact 95% CI [0.02, 0.12]), against **13 of 27 frontier runs** ([0.29, 0.68]). The frontier executors reached 9 of 9 at the **leanest plan file**, v1, which contains no command lines at all. They stayed at 9 of 9 in every column to its right — 45 of 45 in all — so v1g, v1.25, v1.5 and v2 bought them nothing. That saturation should be read against its interval. 9/9 is [0.66, 1.00], and the lower bound is not separated from several local cells. The local executors reached 34 of 36 only at v2 (Figure 2, Table 3).

**Executor class therefore determined how much specification an executor needed, and it mattered most exactly where the plan was thinnest**. "Class" and not "identity": the local arm is 4-bit quantised, at most 35 B, open-weight and served by ollama locally. The frontier arm is unquantised, of undisclosed size, from a single vendor, served over an API and decoded under different sampling parameters. Four variables move together, and this design separates none of them. Quantisation in particular is a live alternative explanation for a model that cannot emit a correct `lofreq call-parallel` flagset. The decisive control — `qwen3.6:27b` at `q8_0` or fp16 against its own `q4_K_M` — was not run. If precision closes a meaningful part of the gap, the finding is about quantisation rather than open weights. Three models from one vendor is also not a sample of frontier models.

**Figure 2.** Plan gradient with reasoning disabled, Jetson AGX Orin, n = 3 seeds per cell. Rows are the twelve benchmark-1 local executors plus the out-of-sample addition `qwen3.8:27b`, thirteen rows in all. The twelve-model gradient totals given in the text exclude `qwen3.8:27b`, which was run separately and is shown here for comparison. Columns run left to right from no plan (Track B) to the most detailed plan (v2). Cell values are the mean tolerant Jaccard score (M3), which on this arm takes only the values 0 and 1 per run.

**Table 3.** Plan gradient, reasoning off, twelve local executors, three seeds per cell. Track B pools three (plan file, track) cells, hence n = 108. Intervals are Clopper–Pearson exact 95% intervals. **The local arm is reported as a count of runs producing the correct call set, not as a mean score**. On all 324 local reasoning-off cells, M3 takes only the values 0.0 and 1.0. A local "mean M3" column would be the perfect-run

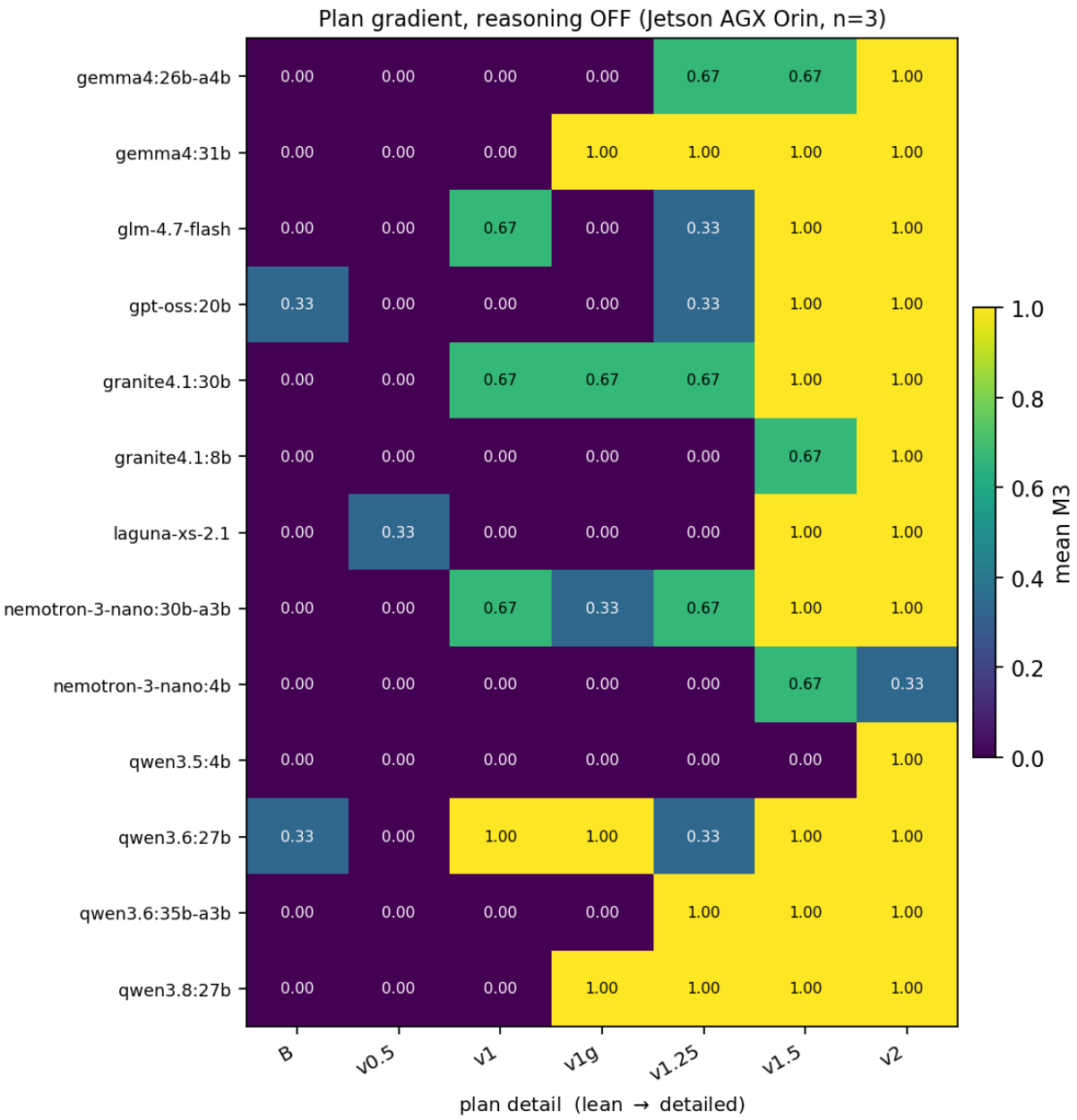


Figure 2: Plan gradient with reasoning disabled

proportion written twice. It would invite the reader to believe partial biological credit was measured, which on this arm it was not. The tolerant Jaccard is retained only for the frontier arm, where it is non-degenerate. Its cells there take the values 0.0, 0.833, 0.938 and 1.0, so a Track-B mean of 0.825 coexists with only 13 of 27 runs perfect. The frontier column pools the three API executors at n = 9 per column, 27 at Track B. Three local conditions — v1g, v1.25 and v1.5 — were later raised to ten seeds per model over thirteen executors (31/126, 63/127 and 109/127; Results, *The prose is close to inert*; Supplementary Table S16). The n = 3 rows are retained here so that the survey is one design.

| Condition | Local runs with the correct call set | 95% CI | Frontier runs with the correct call set | 95% CI | Frontier mean M3 (Jaccard) |
|---|---|---|---|---|---|
| Track B (no plan) | 6/108 | [0.02, 0.12] | 13/27 | [0.29, 0.68] | 0.825 |
| v0.5 (tool order only) | 1/36 | [0.00, 0.15] | 3/9 | [0.08, 0.70] | 0.801 |
| v1 (lean) | 9/36 | [0.12, 0.42] | 9/9 | [0.66, 1.00] | 1.000 |
| v1g (registry-authored) | 9/36 | [0.12, 0.42] | 9/9 | [0.66, 1.00] | 1.000 |
| v1.25 (v1 + one command line) | 15/36 | [0.26, 0.59] | 9/9 | [0.66, 1.00] | 1.000 |
| v1.5 (v2 with prose stripped) | 30/36 | [0.67, 0.94] | 9/9 | [0.66, 1.00] | 1.000 |
| v2 (detailed) | 34/36 | [0.81, 0.99] | 9/9 | [0.66, 1.00] | 1.000 |

**The prose is close to inert; the literal invocations produce the effect**

v1.5 is v2 with all prose stripped, leaving only code fences. It is the **shortest** plan in the set, at 159 words against v2's 660, and it is the second best.

The two halves of the prose-versus-syntax contrast are not equally well resolved. Removing the literal command lines, which is what v1 is, costs 25 of 36 perfect runs (34/36 → 9/36). No sample-size argument touches that. Stripping every word of explanation from v2 costs 4 of 36 (34/36 → 30/36), which is **not resolvable at these sample sizes**: Fisher p = 0.26. Paired by model, the split is 3 improved, 1 worsened, 8 unchanged; sign test p = 0.63. The v1g-versus-v1.25 contrast was unresolved at three seeds (9/36 against 15/36, Fisher p = 0.21) and **is resolved at ten**. Over thirteen executors, v1g is 31/126 (0.246, exact 95% CI [0.17, 0.33]) against v1.25's 63/127 (0.496, [0.41, 0.59]), Fisher exact p = 0.00005. The step from one literal command line to ten is resolved in both designs. v1.25 → v1.5 is 15/36 → 30/36 at three seeds (p = 0.0005). At ten seeds it is 63/127 → 109/127 (0.858, [0.79, 0.91]), $p = 7.1 \times 10^{-10}$. The defensible claim has three parts. Supplying command lines the executor can paste produces the effect. Removing the prose produced no detectable change. The size of the prose term is unknown.

The mechanism is more specific than a count of command lines. v1.25 and v1g each add exactly one literal `lofreq` invocation to v1, and they do opposite things. At ten seeds, v1.25 lifts the local executors to 0.496. v1g leaves them at 0.246, indistinguishable from v1 itself. The difference is *which* line. v1.25 supplies one valid invocation: `lofreq call-parallel --pp-threads 4 -f data/ref/chrM.fa -o results/{sample}.vcf results/{sample}.bam`. v1g supplies a six-line block in the registry's long-form style, with two bare, valueless flags (`--sig`, `--bonf`) that the plan itself then tells the executor it may omit. **The finding is not that more command lines are better. It is that a plan helps when it supplies an invocation the executor can copy verbatim**. v1g is the harder case for the alternative reading. It carries the same invocation as v1.25. It also states the one detail a model is most likely to get wrong: that `results/{sample}.bam` is a positional argument at the end. It still scores at the level of v1, which states the same content in prose. v1g therefore supplies more information than v1.25 and performs half as well. The variable that moves the score is whether the command can be transcribed as written. Verbosity does not predict performance. v1.5 is the shortest plan and the second best. v2 is the longest and the best. v1g is the second longest, and the worst of the four plans containing any command syntax (Table 4). No rank correlation is quoted over five points, two of them tied in both variables. The v1.25-versus-v1g contrast is the argument. Since the seed extension, it is a measured contrast (31/126 against 63/127, Fisher exact p = 0.00005) rather than a two-cell reading.

**Table 4.** Syntax density against outcome, Track A, reasoning off. "Literal command lines" counts non-blank lines inside fenced blocks that invoke a workflow tool or shell construct. That is an operational definition, and the count is sensitive to it. v1g's single `lofreq` invocation is written across six physical lines in the registry's long-form style; it is counted as one, because it is one invocation. v1g's author is the Galaxy IUC registry, extracted mechanically; every other plan was authored by a frontier model. Jetson values are recomputed under this manuscript's retention rule, which scores the one truncated v1g generation 0 and keeps it. They therefore differ in the third decimal from `revision/galaxy_demo/SYNTAX_DENSITY.md`, which drops it. RTX 5080 values come from that file under its own convention. They are given only to show that the ordering reproduces on a second machine. Every frontier cell here is 9/9. The RTX 5080 sheet reports 0.667 in its frontier v1g cell, on a different set of 9 runs; neither figure should be over-read at n = 9. The ten-seed column pools the seed extension over thirteen local executors (Methods; Supplementary Table S16). v1 and v2 were not extended; v2's headline pooling to n = 10 is Table S11.

| Plan | Author | Words | Literal command lines | Local mean M3, Jetson (n = 3/model) | Local mean M3, Jetson (n = 10/model) | Local mean M3, RTX 5080 | Frontier mean M3, Jetson |
|---|---|---|---|---|---|---|---|
| v1 | frontier planner | 422 | 0 | 0.250 (n = 36) | — | 0.211 (n = 38) | 1.000 (n = 9) |
| v1g | Galaxy IUC registry (mechanical) | 558 | 1 | 0.250 (n = 36) | 0.246 (n = 126) | 0.189 (n = 37) | 1.000 (n = 9) |
| v1.25 | frontier planner | 413 | 1 | 0.417 (n = 36) | 0.496 (n = 127) | 0.528 (n = 36) | 1.000 (n = 9) |
| v1.5 | frontier planner | 159 | 10 | 0.833 (n = 36) | 0.858 (n = 127) | 0.892 (n = 37) | 1.000 (n = 9) |
| v2 | frontier planner | 660 | 8 | 0.944 (n = 36) | — | 0.949 (n = 39) | 1.000 (n = 9) |

The right-hand column is what makes the gradient a measurement rather than an assertion. Across the same five plans, the frontier executors score 1.000 everywhere — 45 of 45 — while the local arm climbs from 0.250 to 0.944 on the same files. **Plan detail substitutes for model capability**. The substitution is visible as a difference between two arms run on the same plans, rather than inferred from one arm. The frontier 1.000 is a flat point estimate at n = 9, not an established invariance. v1g is **not** an arm of a planner comparison, which would have to hold syntactic density fixed. It is a mechanically extracted plan at v1-level density, with a non-model author. It behaves as its *content* predicts, not as its authorship or length would. That is useful evidence that the gradient is not an artifact of one planner's house style. This result concedes the transcription objection rather than answering it. The claim that survives is the one a reader can use. **A small local model reliably produces a working pipeline when it is handed invocations it can copy verbatim. The explanation around those invocations adds nothing detectable at these sample sizes**. Whether "copy verbatim" is a figure of speech, or a literal description of what most of these models do, is tested directly below (*A one-path perturbation*).

### How much of the plan is copied

We define a **transcription index**: the token-level recall, in an emitted script, of the 90 literal command tokens of plan v2, after normalising the sample placeholder and the thread count. The corpus is the 514 reasoning-off scripts.

**Table 5.** Transcription index by plan condition, reasoning off, over 514 archived scripts. Recall is of plan v2's 90 literal command tokens, so all conditions are measured against one fixed reference. Brackets are the interquartile range across scripts. The corresponding bootstrap 95% intervals on the mean have a half-width of at most 0.03 in every cell with n > 9, and are omitted for width. The corpus includes the 27 `qwen3.8:27b` scripts that Table 3 excludes; they are 5% of it.

| Condition | Local, all scripts | n | Frontier, all scripts | n | Local, correct runs only | n | Frontier, correct runs only | n |
|---|---|---|---|---|---|---|---|---|
| Track B (no plan) | 0.446 [0.42–0.49] | 117 | 0.520 [0.47–0.56] | 27 | 0.428 | 6 | **0.505** [0.46–0.54] | 13 |
| v0.5 | 0.454 [0.42–0.49] | 39 | 0.501 | 9 | 0.467 | 1 | 0.444 | 3 |
| v1 | 0.515 [0.48–0.54] | 39 | 0.522 | 9 | 0.509 | 9 | 0.522 | 9 |
| v1g | 0.518 [0.48–0.56] | 38 | 0.517 | 9 | 0.504 | 12 | 0.517 | 9 |
| v1.25 | 0.509 [0.49–0.52] | 39 | 0.533 | 9 | 0.501 | 18 | 0.533 | 9 |
| v1.5 | 0.714 [0.70–0.74] | 39 | 0.658 | 9 | 0.717 | 33 | 0.658 | 9 |
| v2 | 0.746 [0.74–0.76] | 122 | 0.695 | 9 | **0.749** [0.74–0.76] | 110 | 0.695 | 9 |

Three things follow. First, the unconditioned Track-B floor is confounded with correctness and is not the derivation baseline. 111 of the 117 local Track-B scripts failed, and a script that died early mechanically has low token recall. The right floor is what a *correct* pipeline built with no plan contains: **0.505**, over the 13 frontier Track-B runs that produced the correct call set. Against that floor, plan-conditioned correct scripts at v2 recover **0.749**. **So about half of v2's literal command tokens are what any correct pipeline for this task contains regardless of plan, and roughly a further quarter is transcription**. Second, **the claim that the index tracks the score is withdrawn**. There is no within-condition signal. At v2, successful scripts recover 0.749 against 0.722 for failed ones. At v1, the figures are 0.509 against 0.517, the wrong direction. Third, local scripts recover 0.956 of v1.5's own tokens. That is near-verbatim reproduction of a plan that is nothing but code fences — the inert-prose result arriving from the artifact side.

What the index cannot settle is what the remaining quarter is. A model that reflows supplied tokens and a model that binds a specification to inputs both score near 0.75. No token count forces them apart. The condition that does is a perturbed plan — v2 verbatim, with the reference moved — because a transcriber reproduces the defect and a binder repairs it. **That condition was run after the second review cycle, and it is the next section**.

### A one-path perturbation separates transcription from binding

The design is in Methods. The plan is v2 verbatim. The reference genome sits at `data/ref/GRCh38_chrM/rCRS.fa` instead of the plan's literal `data/ref/chrM.fa`. The real path is stated in the prompt's DATASET section. The sandbox was validated with a hand-written copier (fails) and binder (succeeds) before any model ran. Thirteen local executors ran at three seeds each, against a control of 34/36 perfect on the unperturbed v2.

**The result is 6 of 39 perfect, and the outcome is bimodal by model: every model is 3/3 or 0/3** (Table 6). This was verified from the emitted scripts, not inferred from the scores. Both surviving models wrote `data/ref/GRCh38_chrM/rCRS.fa` throughout. All eleven failing models wrote the plan's `data/ref/chrM.fa` verbatim; several derived index names such as `chrM.fa.fa` from it. Every failing run died at the first step, with `bwa index` unable to open the file.

**Table 6.** The perturbed-plan pass: plan v2 verbatim, reference moved to a path stated in the prompt, thirteen local executors × 3 seeds, reasoning off. Outcomes read from the emitted scripts and per-cell logs (`matrix_jetson_perturbed.jsonl`; artifacts under `revision/runs_perturbed/`). Control: unperturbed v2, 34/36 perfect.

| Outcome | Models | What every emitted script wrote |
|---|---|---|
| Bound (3/3 perfect) | `gemma4:31b`, `gpt-oss:20b` | the true path `data/ref/GRCh38_chrM/rCRS.fa`, carried into every derived path |

| Outcome | Models | What every emitted script wrote |
|---|---|---|
| Copied (0/3 perfect) | `qwen3.6:27b`, `qwen3.8:27b`, `qwen3.6:35b-a3b`, `gemma4:26b-a4b`, `glm-4.7-flash`, `granite4.1:30b`, `granite4.1:8b`, `laguna-xs-2.1`, `nemotron-3-nano:30b-a3b`, `nemotron-3-nano:4b`, `qwen3.5:4b` | the plan's stale `data/ref/chrM.fa` verbatim; every run failed at the first step (`bwa index`: fail to open file) |

Four things follow, and together they change what the gradient means. **First, for eleven of thirteen local models, v2 success is transcription**. They copy the plan's command lines without reconciling them against the stated inputs. One path mismatch, stated in the prompt they were reading, sent a condition that scores 34/36 to 0. Reviewer 2's objection — that the headline result rewards pasting — is therefore confirmed for most of the model class, as a measurement rather than a concession. **Second, binding exists in this model class but is rare**. Two of thirteen models adjusted every path and produced the correct call set in all three seeds. Binding is therefore demonstrable, not hypothetical — and it is a property of individual models, not of the class. **Third, rank on the unperturbed gradient does not predict binding**. The paper's original headline model, `qwen3.6:27b`, 10/10 at unperturbed v2, copied. `gpt-oss:20b`, one of only three models below 10/10 at the unperturbed headline condition (9/10), bound. **Fourth, the claim that survives is narrower than plan sufficiency: a detailed plan makes most local models work only while its literal paths match the data**. Reliable execution under a stale plan requires either a binding-capable model or a plan kept exactly current. Whether frontier executors bind under the same perturbation is unknown, because no frontier arm was run. And at three seeds per model, a 3/3 is [0.29, 1.00], so the bimodality is sharp but each per-model rate is wide.

### Three attempts separate transcribers that repair from transcribers that do not

The perturbation above is single-pass, so a bounded repair arm re-ran it with feedback. The condition is the perturbed plan, unchanged. If the script exits nonzero, the model sees its own script, the exit code and the last 40 lines of the execution log, and may submit a fix, up to three attempts. The retry signal is the exit code only; the score is computed afterward and never shown. Thirteen local executors ran at three seeds each (`matrix_jetson_repair.jsonl`; per-attempt scripts under `revision/runs_repair/`).

Feedback split the eleven copiers into models that repair and models that do not (Table 7; per-seed attempts and scores are Supplementary Table S18). Three of the eleven copier models repaired. Eight did not, although the error named the missing file and the correct path was in their prompt. Repair rescued 8 of the 33 copier seeds. With repair, the class reached 14 of 39 perfect runs, against 6 of 39 one-shot and 34 of 36 unperturbed. The single-pass constraint therefore accounts for part of the transcription finding, not all of it. The taxonomy is three tiers: bind (2 models), repair (3), neither (8). One retry converted `qwen3.6:27b` and `qwen3.8:27b` from total failure to perfect. For those two models, an execute-and-retry round was worth more than any amount of plan prose.

**Table 7.** The bounded repair arm: the perturbed condition with up to three attempts, the error fed back after each nonzero exit. Thirteen local executors × 3 seeds, reasoning off. Per-model attempts and per-seed scores are Supplementary Table S18.

| Tier | Models | Outcome |
|---|---|---|
| Bind on attempt 1 | `gemma4:31b`, `gpt-oss:20b` | one attempt, perfect, all seeds (the controls; unchanged) |
| Repair on attempt 2 | `qwen3.6:27b`, `qwen3.8:27b` | perfect on attempt 2 in all three seeds |
| Partial repair | `laguna-xs-2.1` | perfect on attempt 2 in two of three seeds |
| No repair | the remaining eight models | 0 of 24 seeds recovered in three attempts |

Three behaviours were verified from the per-attempt scripts, not inferred from the scores. First, `qwen3.6:27b`'s

fix is a genuine repair. Its second attempt tests for the old path, falls back to the path stated in the prompt, and proceeds. It used the prompt's information once the error pointed at the file. Second, `qwen3.6:35b-a3b` resubmitted the identical dead path in all three attempts, with the error naming that path in front of it. Third, `granite4.1:30b` and `gemma4:26b-a4b` guessed `data/ref/rCRS.fa` on attempt 3: the correct filename from the prompt, in the wrong directory. They used part of the prompt's information and did not check the rest. Failure to repair is not failure to act. The non-repairers changed other things — flags, loops, index names — while keeping the dead path. Four caveats bound this arm: three seeds per model, one perturbation type, a retry only on a nonzero exit, and no frontier arm.

### Repeated sampling at the headline condition

The n = 3 headline does not survive repeated sampling. Seven seeds were added to the original three at v2 Track A (Figure 3, Supplementary Table S11). Nine of 12 executors are 10/10 (exact 95% CI [0.69, 1.00]). `gpt-oss:20b` is 9/10; `qwen3.5:4b`, which scored 3/3 at n = 3, is 7/10; `nemotron-3-nano:4b` is 1/10 ([0.00, 0.45]). Every model that moved, moved down. That is the expected direction rather than symmetric noise. We therefore report n = 10 as the headline and treat every n = 3 cell as an interval. The out-of-sample pair `qwen3.8:27b` against `qwen3.6:27b` is underpowered rather than null. 8 of 27 matched cells are discordant, splitting 6 to 2 in favour of the *older* model (p = 0.29). The generational framing has therefore been removed from the abstract and the conclusion.

**Figure 3.** Repeated sampling at the headline condition (plan v2, Track A, reasoning off). Grey points are the n = 3 means as originally published; red points are the pooled n = 10 means. Every model that moved, moved down; no model rose. This is repeated sampling from one configuration, not independent replication.

### The conventional metrics are not degenerate: recall without the exit-status rule sits above run-level success

Over all 324 reasoning-disabled runs, TP = 1,042, FP = 0 and FN = 1,874. Precision is 1.000 by construction, because the truth set is the canonical output of the same deterministic caller. **Recall, however, does not equal the run-level success rate** in any condition except v2. It is 0.086 against 6/108 at Track B, 0.306 against 9/36 at v1, 0.485 against 15/36 at v1.25, and 0.886 against 30/36 at v1.5 (Supplementary Table S12). Pooled, recall is 0.357 against a success rate of 0.321. No pooled F1 is reported: it averages over conditions that share no denominator of interest, and it re-expresses a binary.

**The recall curve computed without the exit-status rule sits above the run-level success curve at every lean condition**. The excess is real biology: variants correctly called by runs the run-level metric scores 0. Two mechanisms produce it. **26 of the 324 runs produced exactly 2 of the 9 truth variants**, concentrated at the lean end, so output is near-bimodal but not strictly so. And **8 of the 324 runs wrote all four VCFs and then exited non-zero**, of which 6 had recovered all nine truth variants. That is why 110 runs recovered the complete truth set while only 104 scored a perfect run. Scoring such a run 0 is defensible — a pipeline that exits non-zero has not finished — but it is a choice. That choice, not the biology, is what makes the run-level metric degenerate. So the conventional metrics quantify how often a model gets the biology right and the plumbing wrong: 8% of runs by partial call sets, plus 2.5% by exit status. What they cannot do is separate a good caller from a bad one. The truth set is the same caller's own output, and no perturbation series that would vary accuracy continuously was run.

### Enabling reasoning does not substitute for plan detail

Over 324 reasoning-disabled and 270 reasoning-enabled cells, restricted to the 10 models runnable in both arms, reasoning helps where the plan is thin and costs accuracy where it is sufficient. The per-plan differences are **+0.17 at v1, +0.23 at v1.25 and −0.20 at v2**. At v2 Track A, 4 of 10 models regress and none improve. Enabling reasoning on `qwen3.8:27b` at plan v1g also needs a budget two to three times larger merely to emit a script. Even then, it falls from 3/3 correct in ~130 s to 1/3 correct in 7,395–12,974 s. **Chain of thought is not a substitute for a sufficient plan, and at the sufficient plan it is a liability**. The reasoning-enabled arm has a 6.7% harness-bounded failure rate against 0.3% disabled. It therefore measures model-plus-budget rather than model capability (Supplementary Results, Figures S2–S3).

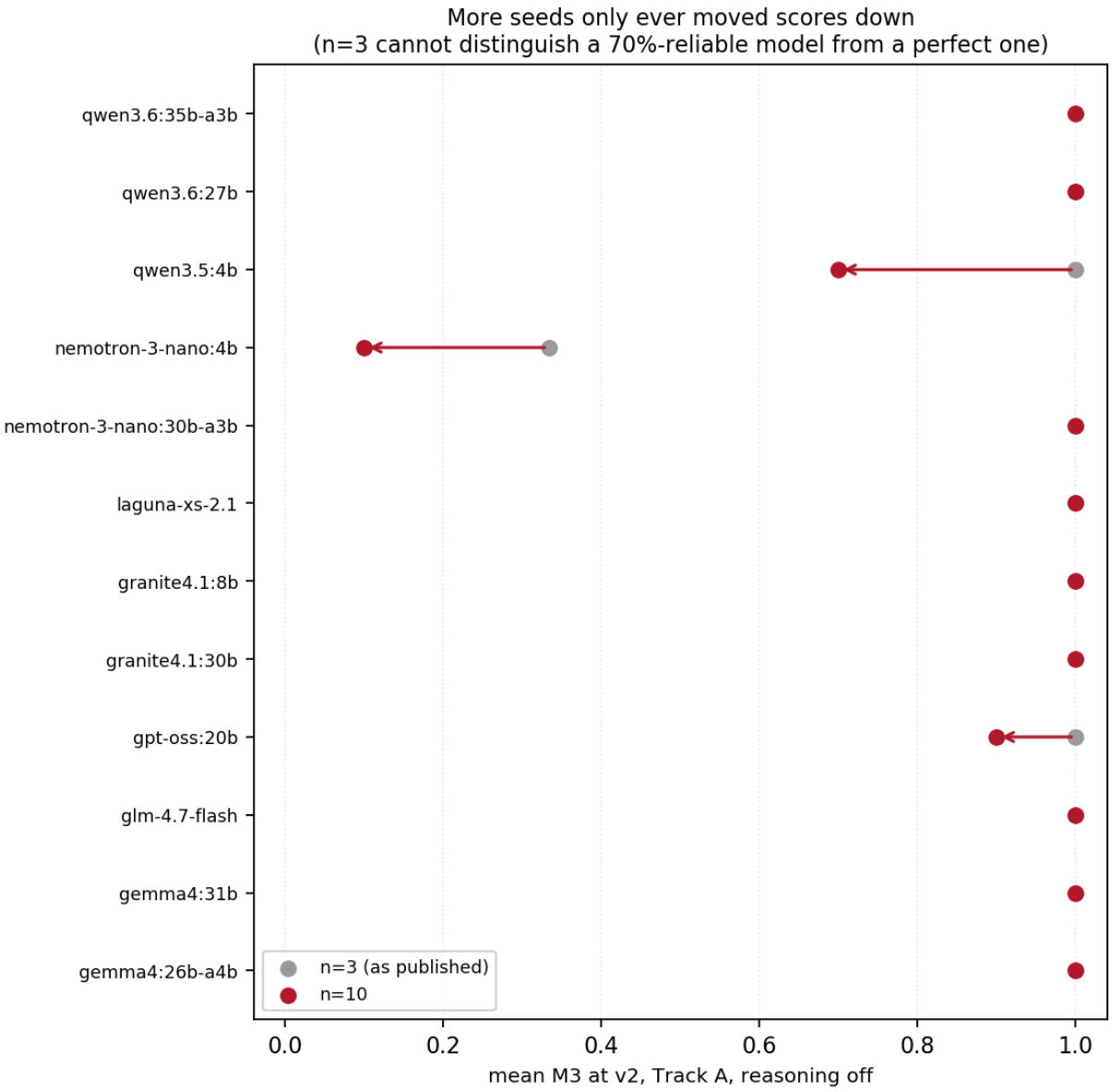


Figure 3: Repeated sampling at the headline condition

**Injected failures: the outcome is set by the plan and the pattern, not by the model**

Splitting the injected matrix along the true-error line changes the reading of the result (Figure 4). For the local executor `qwen3.6:27b` on the baseline v2 plan, the 24 true-error cells give modal handle categories of crash 15, propagate 6 and recover 3. That is **a recovery rate of 12.5% (exact 95% CI [0.03, 0.32]) against a crash rate of 62.5% ([0.41, 0.81])**, and a mean M3 of 0.000. On the 12 non-error cells the script recovers trivially, because the tool succeeded. The pooled figures of 0.417 by handle category and 0.750 by the per-sample recovery flag are therefore not quotable as recovery rates. Nothing here shows that any executor handles injected failures well. These runs had no opportunity to retry. **A 12.5% recovery rate under a single pass is therefore the strongest internal motivation for the repair loop reviewers asked for**. A bounded version of that loop was later run at the perturbed condition (*Three attempts separate transcribers that repair from transcribers that do not*). The honest reading of benchmark 1 is that it measures what a model does without a loop.

The identical distribution obtained for the frontier executor is a degeneracy, not a parity finding. Across the 39 v2 cells, `qwen3.6:27b` and `claude-opus-4-7` agree in **every cell on both handle category and M3**. The same holds under v2_defensive. Five of the six executors produce exactly crash 15 / propagate 6 / recover 3 on the 24 true-error v2 cells. Two architecturally unrelated families do not converge cell-for-cell by shared competence. The handle category at the baseline plan is therefore essentially determined by the injected pattern. Two things bound that reading. The per-cell artifacts for the error matrix were **not archived**, so the script diff that would confirm it cannot be run. And the frontier comparator here is a **different model generation** from the arm that produces the paper's main result. The model term is not zero everywhere. Under v2_defensive, `qwen3.6:27b` and `claude-opus-4-7` resolve the 24 true-error cells to recover 9 and partial 15. `qwen3.6:35b-a3b` resolves them to recover 3 and partial 21, with its mean M3 *falling* to 0.208. `granite4` produces 24 crashes in 24 cells. **So a defensive plan changes error handling, and how much it helps is model-dependent**.

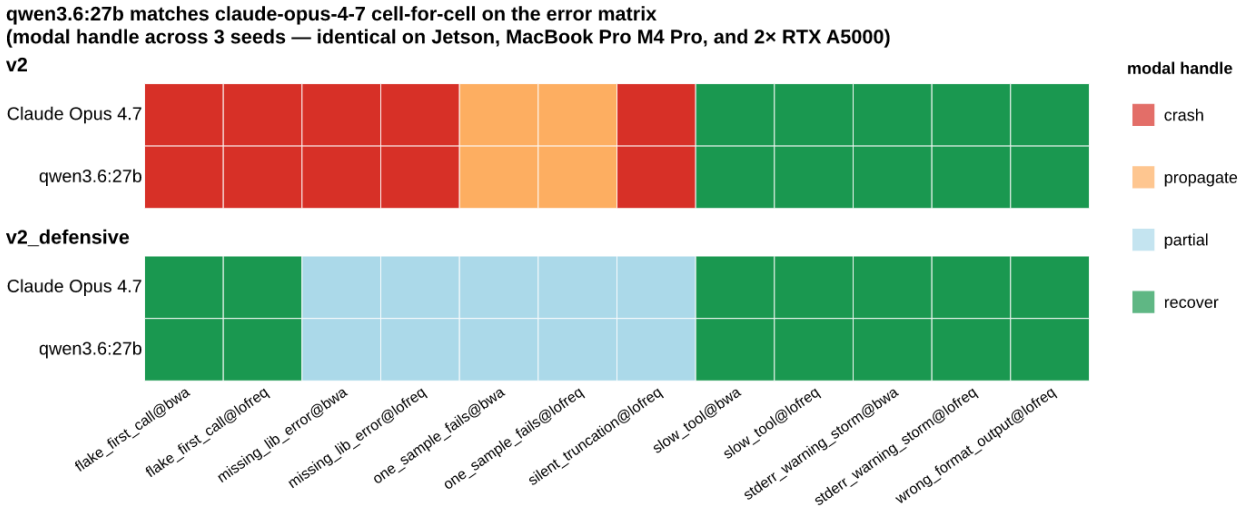


Figure 4: Injected-failure matrix

**Figure 4.** Injected-failure matrix: modal handle category across three seeds, for the local executor `qwen3.6:27b` and the frontier executor `claude-opus-4-7`. Top block: baseline plan v2. Bottom block: plan v2_defensive. The two executors match in all 39 cells of each block. Five of the six executors in the matrix match on the v2 block. That is evidence that the outcome is pattern-determined, not evidence of model parity (see text). Columns 8–11 (`slow_tool@bwa`, `slow_tool@lofreq`, `stderr_warning_storm@bwa`, `stderr_warning_storm@lofreq`) inject no error; they are latency and log-noise tolerance checks, not recovery tests (Supplementary Table S6). The columns are alphabetical, so the twelfth and rightmost, `wrong_format_output@lofreq`, is a true-error pattern, and a green cell there is a genuine recovery.

A lexical audit of the 766 archived `run.sh` files replaces the claim that no destructive or escape behaviour

was observed. None invokes a package manager, network fetch tool or `sudo`. None directs output to an absolute path outside `/dev/` and `/tmp/`. 17 (2.2%) write under `/tmp/`. One script — `rm -f /tmp/*.txt /tmp/temp_*.vcf.gz` — **is a genuinely out-of-sandbox action**, and it came from a no-plan run. The `/tmp/` writes concentrate in no-plan runs generally: 11 of 303 no-plan scripts, against 6 of 463 written with a plan. A model given no plan invents its own scratch-space convention, and the convention it invents leaves the sandbox.

### Case study: a tool-using agent loop on a production server

This is a case study, not an experiment. It has one invocation per arm, no replicates, no frontier control, and a plan repeatedly corrected while it ran. Nothing in it carries a confidence interval. Tool-call and wall-clock figures are transcribed from `FACTS.md`; the counts are re-readable from the public histories.

Given the plan and a live MCP connection to usegalaxy.org, both local executors imported the 30-step IWC `rnaseq-pe` workflow by TRS identifier. Both verified the six paired-end inputs and the annotation. **A human had pre-built both of those after a harness transport fault, so those two plan steps were verification and not construction**. Both executors then submitted a parameter-correct invocation: 2 of 2 inputs and 8 of 8 parameters. The dense arm did this in 38 tool calls over 44 min, the MoE arm in 19 over 22 min. **The 8/8 figure measures transcription, not derivation**, because the invocation step supplied a literal JSON body. The invocations reproduced the published quantification. Both arms yielded 5,594 genes, verified by downloading and counting the counts table rather than by accepting a reported figure. Assigned fragments were 15.7–25.3 M per sample, at 93.1–93.6% of uniquely mapped fragments, against a published ~92% on that denominator (Supplementary Table S14, Figure 5). The two arms produced byte-identical values from twelve distinct dataset identifiers. That is expected from a deterministic server-side workflow on identical inputs, and it is not independent replication of model behaviour.

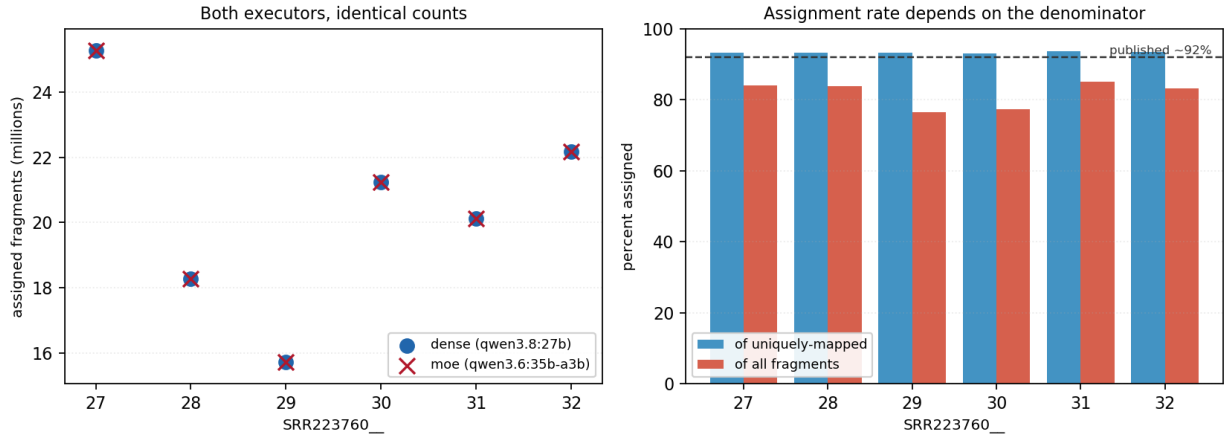


Figure 5: Per-sample featureCounts results

**Figure 5.** Per-sample featureCounts results for the benchmark-2 case study. Left: assigned fragments per run; the dense (`qwen3.8:27b`) and mixture-of-experts (`qwen3.6:35b-a3b`) arms are exactly superimposed. Right: assignment rate against two denominators — uniquely mapped fragments and all fragments — with the published ~92% benchmark marked. The gap between the two denominators is multi-mapping, a property of the data rather than of the executor.

**The result carries three caveats that must accompany it wherever it is claimed**. Both executors submitted a parameter-correct invocation, and the workflow produced benchmark-matching counts. **But the plan supplied the invocation body literally, a human pre-built two of the seven plan steps' inputs after a transport fault, and neither executor completed the final reporting step**. Neither reported a single per-sample value in the two runs of step 7. A human read the six values out of the server (*Task scoping*). "Both executors submitted a correct invocation" is true; "both executors succeeded" is not.

Supervised human time was substantially larger than the agent wall time and was not logged. Reasoning was suppressed in both models. Job-level outcome counts are not reported, because the jobs-summary JSON was not archived.

**Repeated human correction of the plan, and not any change of model, is what produced a clean run**. `FACTS.md` records eleven plan defects, all authored on the planner side, against two executor errors and one harness error. **The ledger and the diffs that fixed each entry are not released**. The attribution therefore cannot be checked, and must be taken on the author's contemporaneous record. That is why the count is used here and not in the Introduction, Discussion or Conclusion, which say "repeated human correction" instead. The ledger, and the five defect classes worth naming for plan authors, are Supplementary Table S15 and its accompanying text. The most expensive defect is the clearest instance: a GFF3 annotation staged where featureCounts requires a GTF carrying `gene_id`. It aborted all six counting jobs two hours in, because the IWC workflow hard-codes the `exon/gene_id` pair and exposes neither as an input. Plan sufficiency was established during this exercise rather than tested by it. The case study contains no plan gradient, so nothing in it discriminates a sufficient plan from an insufficient one.

### Failure modes visible only in the agentic setting

Three failure modes appeared that a single-pass benchmark cannot produce. First, neither executor completed the final reporting step, so a scorer reading only the invocation would record two clean successes. Second, neither model could escape its own bad durable state. One arm re-read its notebook, found the annotation identifier recorded before the GFF3 was replaced, and re-invoked the doomed workflow. That lost about 2 h of correct upstream compute. Durable memory amplifies a stale fact as readily as a correct one. Third, the zero-tool-call empty turn is a recurring mode. 6 of 30 fabrication-study launches (20%) produced no answer file. The observed mode was a session that made zero tool calls, ran about a minute and stopped normally. That mode is invisible to any scorer that reads only the answers produced. None of the three can occur in benchmark 1. As findings they are weak. The first two are properties of `loom` as much as of the executors. The empty-turn rate rests on design arithmetic rather than an instrumented count.

### Task scoping, not capability, decides whether the reporting step happens

The same reporting job was put to the same two executors, against the same finished workflow, in two framings. **The first framing was step 7 of the seven-step plan**, behind a monitoring step with no bound on its duration. There, neither executor produced a single per-sample number, although both fetched the datasets that contain them. The MoE arm used 22 tool calls over 54 min, the dense arm 26 over 24 min. **The second framing was a standalone task**: six dataset identifiers, report `Assigned` for each, cite the identifier beside every number. Both models did that correctly and repeatedly: 118 of 144 sample-level judgements correct, 14 correct abstentions, and no fabricated value.

The models can demonstrably read these datasets and report them. The omission is therefore not a capability limit but a limit of where the step sat. A step of unbounded duration ahead of it consumed the session budget, and nothing in the plan forced the report out first. The consequence for plan authors is a rule, not a caveat. **Make reporting its own task, with its own budget, rather than the last item behind an open-ended wait**. This is the paper's thesis arriving in a third channel, with the weakest evidence of the three. There was one invocation per arm and no second harness, so a scaffold explanation and a `loom` explanation are not separated. The standalone framing is also a different task shape, as well as a different position in the plan.

### Fabrication is a scaffold failure, and the bound is much weaker than a per-judgement count suggests

One fabrication incident occurred. In a resumed session, an executor emitted a fluent, specific failure report: a named trimming tool failing on a named sample at a named error rate, with a retry and a decision to stop. It had made zero tool calls, and its own invocation was healthy, with 48 jobs `ok`. Every element was false. The tool name was harvested from the harness's own injected system context.

A controlled study then measured the rate: three conditions, two models and five seeds against known ground truth. Of the 30 launches, **6 (20%) produced no answer to score**. Among the 24 that did: **0 fabrications in 144 sample-level judgements drawn from 24 scorable sessions**, of which **118 were correct, 14 were correct abstentions and 12 were `ABSENT`**. In the blocked condition, both models wrote `UNAVAILABLE` for exactly the two unreadable identifiers every time. They reported the other four correctly, with identifiers cited. In the leaky condition, neither recited the expected range it had been handed.

The bound must be stated at the session level and conditionally, and the word "never" is not used. The six judgements within a session are not independent. A session that opens the right datasets gets all six right; one that does not gets none right. The effective sample size is therefore 24, not 144. **The rule of three on 24 sessions gives a 95% upper bound of about 12.5% (Clopper–Pearson exact, 0/24: [0, 0.142]), conditional on an answer having been produced, which happened in 24 of 30 launches**. **The model either answered correctly or did not answer at all, and it did not answer one launch in five**. The two halves of that statement must be quoted together. The per-judgement figure of 0/144, upper bound about 2.1%, is roughly five times tighter than the data support. Two limitations govern even the session-level bound. The scored denominator excludes the 6 launches that produced no answer file. That excluded class is **precisely the zero-tool-call class the original incident belongs to**, so the study excludes, at a 20% rate, exactly the regime whose rate it is trying to bound. And every cell is single-shot with a reachable tool path, whereas the incident arose in a resumed session with no reachable path.

### Cost: a local executor does not pay for itself on a task this small

Median wall time per generation on the sufficient plan varies from 29 s on two RTX A5000s to 518 s on a MacBook Air, across the five platforms (Supplementary Table S7). Accuracy was not measured comparably across platforms at these sample sizes. No claim of platform-independent accuracy is therefore made (Supplementary Table S13).

**The cost accounting contradicts this study's own original motivation**. Measured API cost was $0.0662 per run over 81 runs. Measured local wall-clock time was 86.7 s per run, a marginal electricity cost of $0.00016. Per-call inference cost is therefore not the bottleneck, and "free" is replaced throughout by "no marginal API fee". Against a three-year cost of ownership of $5,045, cost recovery arrives at 76,424 runs. That is 15.3 years at 5,000 runs per year, and therefore not a break-even under a three-year horizon. The like-for-like **annualised** comparison gives a fixed local cost of $1,682 per year, equalled by API spend at **25,475 runs per year**. Even that figure should not be quoted bare, because six of the eight inputs are assumptions. Charging labour to the local arm alone is asymmetric, since the API route also needs setup and maintenance. Adding an API-side labour term moves the annualised crossover to 17,900 runs/year at 4 h/year, and 10,325 at 8 h/year. At 16 h/year, the local box is cheaper at any run rate. **The defensible summary is a range with its driver named: the crossover falls between roughly 10,000 annualised runs per year and never, depending on an API-side labour term that was not measured**. The argument survives only by task size. Holding the three-year total fixed, the runs needed to recover it fall to 5,046 at $1.00 per API run — the scale of a multi-step agent loop — and 1,009 at $5.00. **The economic case for a local executor is made by agentic workloads, not by the single-pass task this paper originally rested on** (Supplementary Table S9).

## Discussion

### What was demonstrated and what was not

Given a plan that supplies the literal invocations, free 4-bit open-weight models on a single sub-$2,000 board produced a working per-sample variant-calling pipeline in a single pass. Nine of twelve were perfect on all ten seeds at the most detailed plan condition. Where the plan was thin, the same models produced almost nothing: 6 of 108 no-plan runs correct. The frontier models produced a correct call set in 13 of 27 no-plan runs, and were 9 of 9 from the leanest plan file onwards. The first finding is the requirement for near-executable detail, not its absence. It is about a class of executor rather than model identity, because quantisation, size, vendor, serving stack and decoder all move together between the arms. The second finding says what that detail is doing. Under a one-path perturbation of the same detailed plan, eleven of thirteen

local models copied the stale path and failed at the first step in every seed. Two rebound every path and stayed perfect. **For most of this model class, plan-conditioned success is transcription; binding exists but is a property of individual models, not of the class**.

Five things were not demonstrated. First, autonomous analysis: in both benchmarks the specification was authored elsewhere, and repeated human correction preceded any clean run in the case study. Second, binding as a property of the class. The perturbation resolves the transcription question for this model set, in the direction of transcription for eleven of thirteen. But it does so under one perturbation type, at three seeds. The two binders do not license any claim about local models generally, still less about frontier executors, which were not run on the perturbed plan. Third, frontier-equivalence on the second workflow class, because no frontier executor was run on it. Fourth, the substitution of iteration for plan detail, which bounds the title directly. **The requirement for near-executable plans holds only under this study's single-pass constraint, which is a property of the harness and not of the models**. A bounded repair arm was run at the perturbed condition, with the error fed back and up to three attempts. It answers the question in part. Three of the eleven transcribers fixed the stale path when shown the error, and eight did not. A repair arm at the lean plans v1 and v1.25 — feedback in place of specification — was still not run. Fifth, an external anchor: none of the numbers here sits on a published scale.

**Plan sufficiency is the engineering object**

The specification, not the model, separates success from failure *among models of this class*. The frontier executors did not need the specification. Plan detail therefore substitutes for model capability; it is not that model identity is irrelevant. What makes a plan sufficient is taken from the three failure records. From the gradient: a sufficient plan carries at least one invocation the executor can copy verbatim. Stating the same invocation in prose, or in a form that cannot be run as written, does not help. From the perturbation: for a transcriber — which is what eleven of thirteen local models here are — sufficiency means literal **and current**. The plan's paths must match the data on disk at run time, because a transcriber reproduces them without reconciling them against the stated inputs. A stale detailed plan is therefore worse than a lean one for a transcriber. It fails at the first path, while looking like the condition that scores 34/36. The plan is not a specification for these models; it is the program. The repair arm qualifies that for some models. For `qwen3.6:27b` and `qwen3.8:27b`, one execute-and-retry round replaced the need for a current path: shown the error once, both were perfect on the second attempt in every seed. For eight of the eleven transcribers, feedback did not help. They kept the dead path through three attempts. From the case-study defect ledger: a sufficient plan names the identity of every value carried between steps, rather than saying "record the output id" where three ids exist. It states gotchas where a plausible action fails. It specifies a validation and retry policy wherever a late failure is expensive. It states no expected answers in its verification step. The fabrication result adds a measured corollary: plan sufficiency governs the reporting channel as well as the execution channel. The same job, models and datasets produced 118 correct sample-level judgements out of 144 as a standalone task, and not one per-sample number as step 7 behind an unbounded monitoring step.

The consequence usually drawn from this — that the artifact to version-control and review is the plan — must be stated against an objection these data raise. The best-performing lean plan, v1.5, is 159 words and ten literal command lines, and local scripts reproduce 0.956 of its tokens. **A 159-word artifact that is ten command lines, version-controlled, reviewed and reproduced at 96% fidelity is a shell script**. The Introduction already concedes that a validated shell script beats a language model on any stable pipeline. The comparison that would settle it — a ~20-line deterministic templater scored beside the models on the same inputs and on a perturbed set — was not run. On unperturbed inputs the templater will win. The question is whether the model beats it once the inputs move. That is the only case in which the model does something the template cannot. The perturbed-plan pass now supplies one piece of evidence. When the inputs moved, eleven of thirteen local models behaved exactly as a fixed template would: they reproduced the stale path and failed. Two did what no template can, rebinding the paths and producing the correct call set. **For most of this model class, then, the objection stands: the model adds nothing a template does not. For two models it demonstrably adds the one thing that matters**. The templater itself was still never scored beside them. What survives is narrower. Where a laboratory has already decided to use a language model as executor, the plan determines whether it works. And a plan must be validated against

the executor that will run it, because a defensive plan helped two executors, left a third worse than before, and destroyed a fourth.

**What the split buys once cost is off the table**

At $0.0662 per API run and an annualised crossover above 25,000 runs per year, a local executor does not pay for itself on a task this small. What the split still buys is an abstraction boundary: only a workflow description crosses to the vendor. Under a dbGaP or EGA data use agreement, an IRB transfer restriction or HIPAA, that is the difference between a controlled disclosure and a data transfer. Two qualifications belong beside that claim. The boundary holds for the planner role, and not for this study's frontier *executor* arms, whose prompts carried the dataset manifest across it. And the payload audit shows the boundary holds incompletely even for the planner. No host name, conda prefix or credential appeared in any outbound payload. But every executor payload carried the operator's home directory, exported by the very constraint line that prohibits it. And all three planner meta-prompts carried the study's sample identifiers, the class a data use agreement is most likely to restrict. **The sharpest illustration is this study's own design: the frontier-executor arms, whose prompts carried subject labels for a mother–child pair to a commercial API, would not have been permissible under a controlled-access DUA**. Three deployment recommendations follow from the observed failure modes; the four untested production concerns behind them are in Supplementary Table S20. First, run the executor with a bounded set of tools, and verify every state-changing call by read-back; bound the *inputs* to those tools as well as the verbs. Second, treat durable agent memory as a cache that must be invalidated by whatever process changes the underlying facts. Third, require a dataset or job identifier beside every number an agent reports.

**Scope**

Benchmark 1 measured one workflow class with one dataset. The case study does not extend the plan-sufficiency result, because it contains no plan gradient. What benchmark 2 contributes is three failure modes a single-pass design cannot produce, two of which are confounded with `loom`. A two-point plan gradient in the loop would make it test plan sufficiency instead of illustrating it; that gradient was not run. One class of property is unlikely to transfer without testing, and it defines the boundary of every claim here. **The plan-sufficiency result is conditional on tasks with no data-dependent parameter choices**. It may be in part a restatement of the criterion by which the two tasks were selected. The results support the use of local executors for running pipelines whose plans are literal and current. For the two models shown to bind, the results also support attaching such a plan to inputs that have moved. They say nothing about local executors making analytical decisions.

**Limitations**

The evidentiary base is one controlled benchmark on one workflow class with one dataset, plus one uncontrolled case study. No *data*-perturbation series was run: no variation in depth or allele frequency, no swap of the reference sequence itself, no tool-version change. Such a series is what would let the accuracy metrics separate a good caller from a bad one, rather than a wired pipeline from an unwired one. The perturbed-*plan* pass moved a path, not the biology. The perturbed-plan result carries its own limits. Three seeds per model means every per-model 3/3 or 0/3 is an interval, not a rate. A single perturbation type — one moved reference path — means nothing here says how these models respond to renamed samples, an added sample, permuted steps or a wrong output path. And no frontier arm was run, so whether frontier executors bind under the same perturbation is unknown. The repair arm shares those limits — three seeds per model, one perturbation type, no frontier arm — and adds one of its own: it retried only on a nonzero exit. Three entries in the previous version of this table have since been run. They are the seed extension at the discriminating conditions, one variant of the perturbed plan, and one variant of the repair arm. The repair arm ran at the perturbed condition rather than at the lean plans. All three are now Results. Five further experiments would each resolve a claim this paper states conditionally, and none was run (Table 8).

**Table 8.** Experiments that would each resolve a claim this paper states conditionally. None was run.

| Missing experiment | What it would settle | Approximate cost |
|---|---|---|
| The remaining perturbed-plan variants (renamed samples, extra sample, permuted order, wrong output path), and a frontier arm on the path variant | Whether the transcription/binding split holds beyond a single moved path, and whether frontier executors bind | 13 models × 3 seeds per variant, ~1 h of Jetson generation each |
| Bounded repair at v1 and v1.25, up to 3 rounds with stderr fed back | Whether feedback also substitutes for plan detail at the lean plans; the repair arm that was run tested feedback against a stale path only | 72 base cells, ≤216 generations |
| `top_p` and `top_k` pinned at the headline and discriminating cells | Whether any cross-model ordering survives a common decoder | 156 cells, ~4 GPU-hours |
| A deterministic ~20-line templater scored beside the models on the same inputs and on perturbed inputs | Whether the model beats the baseline the paper concedes usually wins | no GPU |
| `qwen3.6:27b` at `q8_0` or fp16 against its `q4_K_M` self at v1 and v1.25 | Whether the finding is about quantisation rather than open weights | 12 cells |

Three design gaps have no cheap resolution. The planner comparison at matched syntactic density was not run; v1g's density is not matched, and the human-expert and second-frontier-planner arms remain unrun. The expanded error-injection suite was not run as a designed matrix. And benchmark 2 carries no plan gradient.

Three gaps concern the released artifacts and are reproducibility failures rather than limitations of the evidence. The Galaxy plan files, the fabrication study's code and per-cell outcomes, the eleven-item defect ledger with its diffs, and the `loom` session logs are not in the repository. These items rest on `FACTS.md`, and are marked as such at every point of use:

1. The eleven plan defects and their attribution.
2. The two executor errors and the one harness error.
3. The tool-call and wall-clock figures.
4. The fabrication study's cell counts.

Independently verifiable are the per-sample count table and the gene count. Per-cell records carry a plan file path but no plan revision, so command-to-plan-revision provenance is not machine-checkable.

Individual results carry their own limits; those that would change a conclusion if resolved are these. Decoding parameters other than temperature sat at per-model defaults that differ across families, so every cross-model ordering is provisional. The mechanism argument rests on gradient contrasts, and two are resolved at ten seeds: v1g against v1.25, and the step from one command line to ten (Fisher $p = 0.00005$ and $7.1 \times 10^{-10}$). Prose removal remains unresolved, so the size of the prose term is unknown. The residency figures do not reconcile, so no residency number here should be provisioned against. The fabrication bound is ~12.5% at the session level, conditional on an answer being produced in 24 of 30 launches. The economic case fails at these task sizes on the study's own numbers. And the error-injection degeneracy claim rests on per-cell artifacts that were not archived, so it cannot be checked.

Two limitations concern the study as an object. The model set was not stable. The frontier reference moved from Opus 4.7 to Opus 5, which reasons by default and is therefore not a like-for-like substitution. A new Qwen generation was substituted into the benchmark-2 dense arm, in place of the model the frozen rule returned. And the plan author is a closed-API model in both benchmarks, so a laboratory that cannot use one has no evidence here about an open-weight planner. The models named here will be superseded within months. The artifacts are released so the evaluation can be re-run.

## Conclusion

On a 7-step per-sample variant-calling task with no data-dependent parameter choices, twelve free 4-bit open-weight models on a single sub-$2,000 board produced a working pipeline reliably only when the plan supplied the literal invocations. Three frontier models needed only a lean plan with no command lines in it. The frontier models also produced a correct call set about half the time from a problem statement alone. That

contrast establishes executor class, not model identity. The requirement holds under a single-pass constraint. What the literal invocations are doing was then measured directly. Under a one-path perturbation of the detailed plan, the true path was stated in the prompt. Eleven of thirteen local models — the original headline model among them — copied the plan's stale path verbatim. They failed at the first step in every seed. Two rebound every path and stayed perfect. For most of this model class, a detailed plan works by transcription, and only while its literal paths match the data. Binding exists, in two models of thirteen, and rank on the unperturbed gradient does not predict it. A bounded repair arm then re-ran the perturbed condition with the error fed back, up to three attempts. Two transcribers became perfect on the second attempt in every seed, a third in two of three, and eight never recovered. A second workflow class was exercised as a case study on a public production server, where repeated human correction of the plan produced a clean run. Both executors submitted a parameter-correct invocation from a plan that stated the invocation body literally. A human pre-built two of the seven steps' inputs. Neither executor delivered the final report, and a human read the per-sample counts out of the server. For a laboratory, the practical reading is this. Where a language model of this class is used as executor, the plan decides whether it works. The plan must be kept exactly current, because a stale path defeats a transcriber outright. The exceptions are models shown to bind, or to repair when given the error, by a perturbation test like this one. The leanest plan that works here is short and literal enough to be a shell script, a comparison this study did not run. At the task sizes measured here, the reasons to run an executor locally are data governance and freedom from per-call metering, not cost.

## Acknowledgements

This work used the public Galaxy server at usegalaxy.org, maintained by the Galaxy Project, and the community-curated tool wrappers and workflows of the Galaxy Project Intergalactic Utilities Commission and the Intergalactic Workflow Commission. The *Candidozyma auris* RNA-seq data re-quantified in the benchmark-2 case study were generated by Santana et al. (2023) and released under BioProject PRJNA904261. We thank the reviewers of the first version of this manuscript.

An LLM-based coding agent was used for harness construction, analysis scripting and manuscript drafting. The author is responsible for all content.

## References

Alam K, Roy B. From prompt to pipeline: large language models for scientific workflow development in bioinformatics. *arXiv*:2507.20122; 2025. https://arxiv.org/abs/2507.20122

Amstutz P, Crusoe MR, Tijanić N, et al. Common Workflow Language, v1.0. *figshare*; 2016. doi:10.6084/m9.figshare.3115156.v2

Di Tommaso P, Chatzou M, Floden EW, Barja PP, Palumbo E, Notredame C. Nextflow enables reproducible computational workflows. *Nat Biotechnol.* 2017;35(4):316–319. doi:10.1038/nbt.3820

Galaxy Community. The Galaxy platform for accessible, reproducible, and collaborative data analyses: 2024 update. *Nucleic Acids Res.* 2024;52(W1):W83–W94. doi:10.1093/nar/gkae410

Galaxy Project Intergalactic Utilities Commission (IUC). tools-iuc: community-curated Galaxy tool wrappers, commit `39e7456`. GitHub. https://github.com/galaxyproject/tools-iuc Accessed 2026.

Huang K, Zhang S, Wang H, Qu Y, Lu Y, Roohani Y, Li R, Qiu L, Li G, Zhang J, Yin D, Marwaha S, Carter JN, Zhou X, Wheeler M, Bernstein JA, Wang M, He P, Zhou J, Snyder M, Cong L, Regev A, Leskovec J. Biomni: a general-purpose biomedical AI agent. *bioRxiv*; 2025. doi:10.1101/2025.05.30.656746

Kim S, Moon S, Tabrizi R, Lee N, Mahoney MW, Keutzer K, Gholami A. An LLM compiler for parallel function calling. In: *Proceedings of the 41st International Conference on Machine Learning (ICML)*; 2024. arXiv:2312.04511.

Krusche P, Trigg L, Boutros PC, Mason CE, De La Vega FM, Moore BL, Gonzalez-Porta M, Eberle MA, Tezak Z, Lababidi S, Truty R, Asimenos G, Funke B, Fleharty M, Chapman BA, Salit M, Zook JM; Global Alliance

for Genomics and Health Benchmarking Team. Best practices for benchmarking germline small-variant calls in human genomes. *Nat Biotechnol.* 2019;37(5):555–560. doi:10.1038/s41587-019-0054-x

Mehandru N, Hall AK, Melnichenko O, Dubinina Y, Tsirulnikov D, Bamman D, Alaa A, Saponas S, Malladi VS. BioAgents: bridging the gap in bioinformatics analysis with multi-agent systems. *Sci Rep.* 2025;15:39036. doi:10.1038/s41598-025-25919-z

Mitchener L, Laurent JM, Andonian A, Tenmann B, Narayanan S, Wellawatte GP, White A, Sani L, Rodriques SG. BixBench: a comprehensive benchmark for LLM-based agents in computational biology. *arXiv*:2503.00096; 2025. https://arxiv.org/abs/2503.00096

Mölder F, Jablonski KP, Letcher B, Hall MB, Tomkins-Tinch CH, Sochat V, Forster J, Lee S, Twardziok SO, Kanitz A, Wilm A, Holtgrewe M, Rahmann S, Nahnsen S, Köster J. Sustainable data analysis with Snakemake. *F1000Research.* 2021;10:33. doi:10.12688/f1000research.29032.2

Nekrutenko A. Eight LLMs, one RNA-seq dataset: what we learned controlling Galaxy from Orbit. Galaxy Hub news post, 9 June 2026. https://galaxyproject.org/news/2026-06-09-llm-agents-reanalyze-rnaseq/

Santana DJ, Anku JAE, Zhao G, Zarnowski R, Johnson CJ, Hautau H, Visser ND, Ibrahim AS, Andes DR, Nett JE, Singh S, O'Meara TR. A *Candida auris*-specific adhesin, Scf1, governs surface association, colonization, and virulence. *Science.* 2023;381(6665):1461–1467. doi:10.1126/science.adf8972

Shen Y, Song K, Tan X, Li D, Lu W, Zhuang Y. HuggingGPT: solving AI tasks with ChatGPT and its friends in Hugging Face. In: *Advances in Neural Information Processing Systems (NeurIPS) 36*; 2023. arXiv:2303.17580.

Shinn N, Cassano F, Berman E, Gopinath A, Narasimhan K, Yao S. Reflexion: language agents with verbal reinforcement learning. In: *Advances in Neural Information Processing Systems (NeurIPS) 36*; 2023. arXiv:2303.11366.

Su H, Long W, Zhang Y. BioMaster: multi-agent system for automated bioinformatics analysis workflow. *bioRxiv*; 2025. doi:10.1101/2025.01.23.634608

Wang L, Xu W, Lan Y, Hu Z, Lan Y, Lee RK-W, Lim E-P. Plan-and-Solve prompting: improving zero-shot chain-of-thought reasoning by large language models. In: *Proceedings of the 61st Annual Meeting of the Association for Computational Linguistics (ACL)*; 2023:2609–2634. arXiv:2305.04091.

Yao S, Zhao J, Yu D, Du N, Shafran I, Narasimhan K, Cao Y. ReAct: synergizing reasoning and acting in language models. In: *International Conference on Learning Representations (ICLR)*; 2023. arXiv:2210.03629.

Zhou D, Schärli N, Hou L, Wei J, Scales N, Wang X, Schuurmans D, Cui C, Bousquet O, Le Q, Chi E. Least-to-most prompting enables complex reasoning in large language models. In: *International Conference on Learning Representations (ICLR)*; 2023. arXiv:2205.10625.

## Supplementary material

Supplementary Methods, Supplementary Results, Supplementary Tables S1–S20 and Supplementary Figures S1–S4 are in `supplement_R2.md`.